

Operations & Deployment Guide

Azure hosting, CI/CD, and runtime operations

Document	3 of 9
Audience	DevOps / Automation Engineer
Revision	Rev 1.0 · 2026
Status	Handoff

Operations & Deployment Guide — Castillo Planset QC

Audience: the DevOps / Automation Engineer **Scope:** Azure topology, CI/CD, manual deploys, secrets & app settings, the SQLite-on-Azure-Files persistence model, known operational flakes, and monitoring.

This is the definitive operations reference for running the Castillo Planset QC application (an AI-assisted QA/QC service for solar PV plansets: a FastAPI backend that also serves the built React/Vite SPA from a single container). Everything below is drawn from the live infrastructure-as-code (`infra/main.bicep`, `infra/main.parameters.json`), the deployment workflows (`.github/workflows/deploy.yml`, `deploy-staging.yml`), the container image (`Dockerfile`), and the application modules (`backend/app/main.py`, `db.py`, `jobs.py`, `auth.py`, `gemini_client.py`).

1. Azure Topology

1.1 Runtime model

The entire product runs as **one Linux container** on **Azure Container Apps**. Inside that container, FastAPI (`uvicorn app.main:app`) serves:

- the JSON API under `/api/`,
- static run artifacts under `/artifacts` (mounted from the data directory), and
- the compiled React SPA at `/` (mounted only when `FRONTEND_DIST` points at a built frontend — it does, in the image).

Container Apps is used **instead of App Service** because the subscription has **0 App Service VM quota**; Container Apps draws from a separate quota pool. Capabilities are equivalent for this single-container workload.

1.2 Environments

There are two Container Apps, both fed from the same registry and OIDC identity:

Environment	Container App	Branch → deploys it	Image tags
Production	castillo-qaqc-automation	master	planset-qc:<git-sha> + planset-qc:latest
Sandbox / staging	castillo-qaqc-sandbox	staging	planset-qc:staging- <git-sha>

Both live in resource group ``castillo-qaqc-automation-rg`` (region **East US**). The staging workflow reuses the same `AZURE_RESOURCE_GROUP` and `ACR_NAME` secrets, so sandbox shares the RG and registry

with prod but is a **separate Container App revision stream** — experimental builds never touch production traffic.

1.3 Provisioned resources (from `infra/main.bicep`)

Resource	Name (prod)	Notes
Container App	<code>castillo-qaqc-automation</code>	Single always-on replica
Container Apps managed environment	<code>castillo-qaqc-automation-env</code>	Logs → Log Analytics
Container Registry (ACR)	<code>castilloqaqcautomationacr</code>	Referenced as <code>existing</code> ; created before the app
Storage account	<code>st<uniqueString></code>	Azure Files share <code>data</code> , 100 GB quota, SMB, <code>Standard_LRS</code> , TLS 1.2 min, no public blob
Environment storage link	<code>datamount</code>	Binds the <code>data</code> share to the environment (ReadWrite)
Log Analytics workspace	<code>castillo-qaqc-automation-logs</code>	<code>PerGB2018</code> , 30-day retention
User-assigned managed identity	<code>castillo-qaqc-automation-id</code>	Granted AcrPull on the registry

The Bicep template also provisions the Entra **authConfig** resource (see §5) so a full template redeploy cannot silently drop the sign-in gate.

1.4 Container image & runtime configuration

The image (`Dockerfile` at repo root) is a two-stage build:

1. ``node:22-slim`` — `npm ci + npm run build` the frontend to `/build/dist`.
2. ``python:3.11-slim`` — installs `backend/requirements.txt`, copies `backend/`, copies the built SPA to `/app/frontend_dist`, and runs `uvicorn`.

Baked-in image defaults:

```
ENV FRONTEND_DIST=/app/frontend_dist \
    PLANSET_DATA_DIR=/home/data \
    AI_PROVIDER=openai
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

1.5 Ingress, compute, and scale

From the Container App template:

Setting	Value	Why
---------	-------	-----

Ingress	<code>external: true, targetPort: 8000, transport: auto, allowInsecure: false</code>	Public HTTPS; targets the unicorn port
CPU / memory	2.0 vCPU / 4.0Gi	Consumption plan requires memory = 2x vCPU
Revisions mode	Single	New revision takes 100% of traffic
Scale (prod)	<code>minReplicas: 1, maxReplicas: 1</code>	Hard requirement — see below

Single replica is a correctness constraint, not a cost choice. SQLite lives on the SMB share and is opened with a single-writer strategy (§6). **Do not raise `maxReplicas` above 1 on prod.** To scale horizontally you must first migrate the DB to a networked engine (e.g. Azure Database for PostgreSQL) and artifacts to Blob Storage.

Sandbox may scale to zero. Because sandbox is intermittently used and is still SQLite-single-writer safe (0 or 1 replica is never >1), it is appropriate to set `minReplicas: 0` there to pay nothing while idle, at the cost of a ~30–60 s cold start on the first request:

```
az containerapp update -n castillo-qaqc-sandbox -g castillo-qaqc-automation-rg `
  --min-replicas 0 --max-replicas 1
```

Never set `maxReplicas > 1` on **either** app.

2. CI/CD Pipelines

Both pipelines authenticate to Azure with **OIDC federated credentials** — there is no stored publish profile, password, or long-lived service-principal secret. The federated app registration (`github-castillo-qaqc-deploy`) trusts `refs/heads/master` and `refs/heads/staging`, and holds **Contributor** on the resource group (covers both `az acr build` and `az containerapp update`).

Shared environment on both workflows:

```
env:
  IMAGE: planset-qc
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: 'true' # run checkout/azure-login on Node 24
permissions:
  id-token: write # required for azure/login OIDC
  contents: read
```

2.1 Production — `.github/workflows/deploy.yml`

Triggers: `push` to `master`, or manual `workflow_dispatch`.

Steps:

3. `actions/checkout@v4`
4. `azure/login@v2` (OIDC) using `AZURE_CLIENT_ID` / `AZURE_TENANT_ID` / `AZURE_SUBSCRIPTION_ID`
5. **Build in ACR** — builds the image server-side and pushes two tags: ``bash az acr build \ -- registry "$ACR_NAME" \ --image "planset-qc:${{ github.sha }}" \ --image "planset-qc:latest" \ .``
6. **Roll the app** — creates a new revision on the immutable SHA tag: ``bash az containerapp update \ --name "$AZURE_CONTAINERAPP_NAME" \ --resource-group "$AZURE_RESOURCE_GROUP" \ --image "$ACR_NAME.azurecr.io/planset-qc:${{ github.sha }}"``

2.2 Sandbox — .github/workflows/deploy-staging.yml

Triggers: `push` to `staging`, or manual `workflow_dispatch`.

Mirrors prod but:

- image is tagged ``staging-<sha>`` (never collides with prod tags, and does **not** move `latest`);
- the target app name is inlined (`SANDBOX_APP: castillo-qaqc-sandbox`) rather than read from a secret, since it is not sensitive;
- reuses the same OIDC secrets plus `AZURE_RESOURCE_GROUP` and `ACR_NAME`.

```
az acr build --registry "$ACR_NAME" --image "planset-qc:staging-${{ github.sha }}" .
az containerapp update --name castillo-qaqc-sandbox \
  --resource-group "$AZURE_RESOURCE_GROUP" \
  --image "$ACR_NAME.azurecr.io/planset-qc:staging-${{ github.sha }}"
```

2.3 Required GitHub repo secrets

Secret	Used by	Purpose
<code>AZURE_CLIENT_ID</code>	both	OIDC app (client) id for <code>azure/login</code>
<code>AZURE_TENANT_ID</code>	both	Entra tenant for login
<code>AZURE_SUBSCRIPTION_ID</code>	both	Target subscription
<code>AZURE_RESOURCE_GROUP</code>	both	RG for <code>acr build</code> + <code>containerapp update</code>
<code>ACR_NAME</code>	both	Registry name (<code>castilloqaqcautomationacr</code>)
<code>AZURE_CONTAINERAPP_NAME</code>	prod only	Prod app name (sandbox is inlined)

Note: the CI path (`az containerapp update --image`) **only rolls the image** — it never touches the Entra auth config, secrets, env vars, or scale rules. Those are managed out of band (§4, §5) or via a full Bicep redeploy.

3. Manual & Rollback Deploys

Set your working variables once:

```
$RG = "castillo-qaqc-automation-rg"
$APP = "castillo-qaqc-automation"      # or castillo-qaqc-sandbox
$ACR = "castilloqaqcautomationacr"
```

Build and roll manually (equivalent to CI):

```
az acr build -r $ACR -t "planset-qc:manual-$(git rev-parse --short HEAD)" .
az containerapp update -n $APP -g $RG `
  --image "$ACR.azurecr.io/planset-qc:manual-$(git rev-parse --short HEAD)"
```

Roll back to any prior image tag (revisions are immutable and tagged by SHA):

```
az containerapp update -n $APP -g $RG --image "$ACR.azurecr.io/planset-qc:<old-sha>"
```

Full infrastructure (re)deploy — applies scale, ingress, env vars, and the Entra auth wiring from Bicep. Secrets are passed at deploy time and never committed:

```
az deployment group create -g $RG -f infra/main.bicep `
  -p infra/main.parameters.json `
  -p openAiApiKey="<OPENAI_KEY>" authClientSecret="<ENTRA_CLIENT_SECRET>"
```

Windows gotcha: `az acr build`'s log streamer can crash with `UnicodeEncodeError: charmap`. The **server-side build still completes** — verify with `az acr task list-runs -r $ACR --top 1 -o table` and proceed once it shows **Succeeded**.

4. Secrets & Application Settings

4.1 Container App secrets (encrypted, referenced by name)

Secret name	Consumed by	Notes
<code>openai-api-key</code>	The app, via env <code>OPENAI_API_KEY</code> (<code>secretRef</code>)	The AI provider key
<code>microsoft-provider-</code>	The EasyAuth sidecar only	The Entra client secret. The

authentication-secret		application code never reads this — it is used by the platform auth layer to complete the OIDC flow. Present only when <code>enableEntraAuth = true</code> .
-----------------------	--	---

Rotate the OpenAI key and restart the current revision:

```
az containerapp secret set -n $APP -g $RG --secrets openai-api-key=<new-key>
az containerapp revision restart -n $APP -g $RG `
  --revision $(az containerapp show -n $APP -g $RG --query properties.latestRevisionName -o
  tsv)
```

Never print, log, or commit secret values. Use `secretRef` in env, never inline literals.

4.2 Plain environment variables (set in Bicep template)

Env var	Value	Meaning
AI_PROVIDER	openai	Selects the OpenAI branch of the provider abstraction (<code>gemini_client.py</code>)
OPENAI_MODEL	gpt-5.4-mini	Per-page vision/model calls
OPENAI_MODEL_DEEP	gpt-5.4	Deep multi-page reasoning pass
OPENAI_API_KEY	secretRef: openai-api-key	Injected from the secret
PLANSET_DATA_DIR	/home/data	Root of all persistent state (§6)
FRONTEND_DIST	/app/frontend_dist	Enables the SPA mount at /

4.3 Optional / tunable env vars (read by the code, not all set in prod)

Env var	Default	Effect
ANALYZE_CONCURRENCY	2	Max concurrent analyses in the bounded queue (<code>jobs.py</code>). Keep low — one replica shares one OpenAI key and CPU; too many parallel deep runs thrash memory and risk rate limits.
OPENAI_SEED	unset	Best-effort determinism: passes a chat-completion <code>seed</code> for reproducible before/after diffs. Best-effort only — an OpenAI

		model/infra change (new <code>system_fingerprint</code>) can still shift output. Unset = normal sampling.
<code>AI_TIMEOUT</code>	<code>90</code>	Per-request AI timeout (seconds)
<code>AI_RETRIES</code>	<code>2</code>	AI request retry count
<code>GEMINI_API_KEY</code> / <code>GEMINI_MODEL</code> / <code>GEMINI_MODEL_DEEP</code>	unset / <code>gemini-2.5-flash</code>	Only used when <code>AI_PROVIDER=gemini</code>
<code>DEV_USER_EMAIL</code>	unset	Local dev only — identity fallback when the EasyAuth headers are absent (<code>auth.py</code>). Do not set in prod.

5. Authentication (EasyAuth / Entra)

Access is gated by **Container Apps built-in auth** in front of the whole app — no application code performs the login. It is provisioned as a `Microsoft.App/containerApps/authConfigs` resource (`enableEntraAuth`, default `true`), so a full Bicep redeploy preserves the gate:

- Provider: `azureactivedirectory`
- `unauthenticatedClientAction`: `RedirectToLoginPage` — anonymous browsers are bounced to the Microsoft login
- **Single-tenant** (org-only): `sign-in-audience` `AzureADMyOrg`, issuer `https://login.microsoftonline.com/<tenant>/v2.0`
- `authClientId` and `authTenantId` are public identifiers, filled into `infra/main.parameters.json`
- The **client secret** is passed as the secure `authClientSecret` Bicep parameter → stored as the `microsoft-provider-authentication-secret` Container App secret → consumed by the auth sidecar only

Once authenticated, the sidecar **injects trustworthy identity headers** (it strips any client-supplied copies), which the backend reads in `auth.py`:

- `X-MS-CLIENT-PRINCIPAL-NAME`
- `X-MS-CLIENT-PRINCIPAL-ID`
- `X-MS-CLIENT-PRINCIPAL` (base64 claims blob)

`GET /api/me` returns the resolved `{email, user_id, name}`. Run attribution (`created_by`, `created_by_id`) is stamped from this identity, not from the free-text form field.

One-time app-registration setup (needs the app FQDN for the redirect URI) is documented in `DEPLOYMENT.md`; the redirect URI is `https://<fqdn>/.auth/login/aad/callback`. If the secret is lost, reset it and redeploy the template:

```
az ad app credential reset --id <authClientId> --query password -o tsv
# then re-run the `az deployment group create ... authClientSecret=<new>` from §3
```

To later restrict from whole-tenant to named users: Entra → Enterprise applications → this app → Properties → **Assignment required = Yes**, then add users/groups.

6. Persistence: SQLite on Azure Files

6.1 Layout

Everything persistent lives under `PLANSET_DATA_DIR=/home/data`, which is the **Azure Files SMB share `data`** mounted into the container. It survives revisions, restarts, and redeloys.

Path	Contents
<code>/home/data/planset_qc.sqlite3</code>	The SQLite database (<code>runs</code> , <code>issues</code> , <code>projects</code> , <code>feedback</code> tables)
<code>/home/data/runs/<upload_id>/</code>	Uploaded source PDFs + per-run artifacts
<code>/home/data/exports/</code>	Generated XLSX exports
<code>/home/data/logs/planset_qc.log</code>	Rotating app log (5 MB × 5 files)

6.2 The single-writer locking strategy

SMB shares do **not** support the POSIX byte-range locks SQLite normally relies on, so on a share SQLite fails with "database is locked" even for one writer. `db.py` works around this by:

- opening the DB with `file:...?nolock=1` (the unix-none VFS, skipping OS file locking), and
- serialising **every** DB access through a single in-process `threading.RLock`.

This is **correct only for one process / one replica**. It is the reason prod is pinned to exactly one always-on replica. Any move to multiple replicas requires replacing SQLite with a networked DB server first.

Schema is created/migrated idempotently at startup (`init_db()` on the FastAPI `startup` event) using additive `ALTER TABLE ... ADD COLUMN` guards — safe to run against an existing populated DB.

6.3 Backup / snapshot

Because the DB is a single file on the share, back it up by copying it. **Always snapshot before a schema-affecting deploy or migration.**

Option A — download the DB file locally (quiesce first if possible; the app is single-writer so a copy taken between writes is consistent):

```
$STG = az storage account list -g $RG --query "[?starts_with(name,'st')].name" -o tsv
$KEY = az storage account keys list -g $RG -n $STG --query "[0].value" -o tsv
az storage file download --account-name $STG --account-key $KEY `
  --share-name data --path planset_qc.sqlite3 `
  --dest "./planset_qc.${Get-Date -Format yyyyMMdd-HH:mm:ss}.sqlite3"
```

Option B — Azure Files share snapshot (point-in-time, server-side, captures the whole **data** tree including PDFs and exports):

```
az storage share snapshot --account-name $STG --account-key $KEY --name data
# list snapshots:
az storage share list --account-name $STG --account-key $KEY --include-snapshots -o table
```

Consider a scheduled share snapshot (e.g. daily, on a weekday cadence) for routine protection; snapshots are incremental and cheap.

7. Known Operational Flakes

7.1 Docker Hub anonymous-pull rate limit (recurring)

az acr build builds the image from the two Docker Hub base images (**node:22-slim**, **python:3.11-slim**). ACR pulls these **anonymously**, so builds intermittently fail partway with a Docker Hub **`toomanyrequests` / anonymous pull rate-limit** error. This is transient and not a code problem.

Remedy: simply **re-run the failed GitHub Actions job** (Actions → the failed run → **Re-run failed jobs**), or re-trigger via **workflow_dispatch**. It almost always succeeds on retry once the anonymous-pull window resets. For a durable fix, import the base images into ACR and pull them from there, or configure an authenticated Docker Hub credential for the build.

7.2 **az acr build** Unicode log crash on Windows

Covered in §3 — the client-side log streamer crashes but the server-side build completes. Verify with **az acr task list-runs**.

8. Monitoring, Logs & Access

8.1 Container / platform logs (Log Analytics)

```
# Tail recent console output
az containerapp logs show -n $APP -g $RG --type console --tail 100

# Live follow
az containerapp logs show -n $APP -g $RG --type console --follow

# System (platform) events – image pulls, scaling, health
az containerapp logs show -n $APP -g $RG --type system --tail 60
```

The app emits an access log line per request (skipping the noisy `/api/progress/*` and `/artifacts/*` polling paths) and structured module logs.

8.2 Application file log

The app also writes a rotating file log to the share at `/home/data/logs/planset_qc.log` (5 MB per file, last 5 kept). This survives restarts and is useful for post-mortem inspection even after container logs age out of Log Analytics.

8.3 Health & queue introspection

Endpoint	Purpose
GET <code>/api/health</code>	Liveness — returns <code>{"ok": true}</code>
GET <code>/api/runs</code>	Returns <code>[]</code> on a fresh healthy instance; confirms DB reachability
GET <code>/api/jobs</code>	Shared activity feed: queued/running/recently-finished analyses plus <code>{concurrency, queued, running}</code> stats from the bounded queue

Quick health check:

```
$URL = "https://$(az containerapp show -n $APP -g $RG --query
properties.configuration.ingress.fqdn -o tsv)"
curl "$URL/api/health"
```

8.4 Exec into the running container

For live inspection of the share, DB, or logs:

```
az containerapp exec -n $APP -g $RG --command /bin/sh
# inside: ls -la /home/data ; ls -la /home/data/logs ; tail -n 50
/home/data/logs/planset_qc.log
```

Behind EasyAuth, `curl` against `/api/*` from outside will be redirected to the Microsoft login. Use `az containerapp exec` (localhost, inside the auth boundary) to hit the API unauthenticated for diagnostics, or use `/api/health` which returns before any identity dependency.

8.5 Revisions

```
# Which revision is live, and is it healthy?
az containerapp revision list -n $APP -g $RG -o table

# Restart the current revision (e.g. after a secret rotation)
az containerapp revision restart -n $APP -g $RG `
  --revision $(az containerapp show -n $APP -g $RG --query properties.latestRevisionName -o
  tsv)
```

9. In-Memory State Caveat (operational awareness)

The analysis job queue and registry (`jobs.py`) and per-upload progress are **in-process, single-replica** state. A revision roll or restart **drops in-flight and queued jobs** and the recent activity feed. Practically: avoid rolling a new revision while analyses are actively running (watch `GET /api/jobs`), or accept that in-flight work must be re-submitted after the roll. Completed runs are already persisted to SQLite and are unaffected. Like the DB locking model, this queue must move to a shared backing (DB or a real queue) before the app can run more than one replica.

Quick Reference — Variables

```
$RG    = "castillo-qaqc-automation-rg"
$PROD  = "castillo-qaqc-automation"
$SBX   = "castillo-qaqc-sandbox"
$ACR   = "castilloqaqcautomationacr"
$ENV_  = "castillo-qaqc-automation-env"
# Data: Azure Files share `data` -> mounted at /home/data (SQLite + PDFs + exports +
logs)
# Ports: ingress external, targetPort 8000
# Scale: prod min=max=1 (never raise); sandbox may min=0
```